CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 6

Name: Prionti Das | **UID: 24BAI71005**

Branch: CSE(AIML) | **Section/Group: 24AIT_NTPP-3[B]**

Semester: 5th | **Date of Performance:**

Subject: Full Stack -II | **Subject Code: 24CSP-337**

EXPERIMENT – 6.1

1. **AIM:** To implement paginated and sortable REST APIs for efficient and scalable data retrieval.

2. **OBJECTIVE:**
   1. To understand the concept of pagination and sorting in RESTful APIs.
   2. To implement paginated data retrieval using Spring Boot.
   3. To implement sorting of records based on different fields.
   4. To reduce the amount of data returned in a single API request.
   5. To improve API performance and response efficiency for large datasets.
   6. To implement dynamic page number, page size, and sorting parameters.
   7. To design scalable REST APIs for efficient data retrieval.
   8. To test and verify paginated and sortable API responses using Postman.

3. **SOFTWARE REQUIREMENTS:**
   1. Java JDK 17
   2. Spring Boot
   3. Visual Studio Code
   4. Postman

4. **THEORY :**

Pagination is a technique used to divide a large amount of data into smaller pages so that only a limited number of records are returned in each API request. This reduces response size and improves API performance.

Sorting is used to arrange retrieved records according to a specific field, such as **ID, name, or date**, in ascending or descending order.

Spring Boot provides pagination and sorting support through **Spring Data JPA**, using interfaces such as `Pageable`, `Page`, and `Sort`. Parameters such as **page number, page size, sort field, and sort direction** can be provided dynamically through the REST API.

Prionti_24BAI71005

Paginated and sortable APIs are useful for handling large datasets efficiently and building **scalable and responsive applications**.

5. IMPLEMENTATION/PROCEDURE :

1. Create a **Spring Boot application** for implementing paginated and sortable student data retrieval.
2. Configure the project with the required dependencies, including **Spring Web, Spring Data JPA, and H2 Database**.
3. Create a **Student entity** containing fields such as ID, Name, Email, Course, and Age.
4. Create a **Student Repository** by extending `JpaRepository` to access student records from the database.
5. Create a **Student Service** to handle student data retrieval and pagination and sorting logic.
6. Create a **REST Controller** to expose the student data through API endpoints.
7. Implement pagination using Spring Data JPA's `Pageable` and `Page` interfaces.
8. Configure the API to accept pagination parameters such as **page number** and **records per page**.
9. Implement sorting functionality using the **sort field** and **sort direction** parameters.
10. Create a **Student Data Dashboard** to display the paginated student records in a user-friendly interface.
11. Display summary information such as **Total Students, Total Pages, Current Page, and Records Per Page** on the dashboard.
12. Add controls for selecting:
    - **Sort By** field
    - **Order** (Ascending/Descending)
    - **Records per Page**
13. Implement a **search field** to search student records available on the current page.
14. Implement a **Refresh** button to retrieve the latest data from the REST API.
15. Display student records in a table containing **ID, Name, Email, Course, and Age**.
16. Add **Previous, page number, and Next** navigation buttons to move between different pages of records.
17. Run the Spring Boot application and connect the dashboard with the REST API.
18. Test different page sizes and sorting options to verify that records are displayed correctly.
19. Verify that the API efficiently returns only the required records for the selected page.
20. Capture the final **Student Data Dashboard** output showing pagination, sorting, and student records.

6. CODE :

PaginationApiApplicationTest.java:

```
package com.example.pagination_api;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class PaginationApiApplicationTests {
```

Prionti_24BAI71005

```
    @Test
    void contextLoads() {
    }
}
```

Index.html:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Student Data Dashboard</title>

  <link rel="stylesheet" href="style.css">
</head>

<body>

  <div class="container">

    <!-- Header -->
    <header class="header">

      <div>
        <h1>Student Data Dashboard</h1>
        <p>Paginated & Sortable REST API</p>
      </div>

      <div class="api-status">
        <span id="statusDot"></span>
        <span id="statusText">Checking API...</span>
      </div>

    </header>

    <!-- Statistics -->
    <section class="stats">
```

Prionti_24BAI71005

```
        <div class="stat-card">
          <div class="stat-icon">👥</div>
          <div>
            <p>Total Students</p>
            <h2 id="totalStudents">0</h2>
          </div>
        </div>

        <div class="stat-card">
          <div class="stat-icon">📄</div>
          <div>
            <p>Total Pages</p>
            <h2 id="totalPages">0</h2>
          </div>
        </div>

        <div class="stat-card">
          <div class="stat-icon">📑</div>
          <div>
            <p>Current Page</p>
            <h2 id="currentPage">1</h2>
          </div>
        </div>

        <div class="stat-card">
          <div class="stat-icon">🔢</div>
          <div>
            <p>Records Per Page</p>
            <h2 id="recordsPerPage">5</h2>
          </div>
        </div>

      </section>

      <!-- Controls -->
      <section class="controls">

        <div class="control-group">
```

Prionti_24BAI71005

```
    <label for="sortBy">
      Sort By
    </label>

    <select id="sortBy">

      <option value="id">ID</option>
      <option value="name">Name</option>
      <option value="email">Email</option>
      <option value="course">Course</option>
      <option value="age">Age</option>

    </select>

  </div>

  <div class="control-group">

    <label for="direction">
      Order
    </label>

    <select id="direction">

      <option value="asc">
        Ascending ↑
      </option>

      <option value="desc">
        Descending ↓
      </option>

    </select>

  </div>

  <div class="control-group">

    <label for="pageSize">
```

Prionti_24BAI71005

```
        Records
      </label>

      <select id="pageSize">

        <option value="3">3</option>

        <option value="5" selected>
          5
        </option>

        <option value="10">
          10
        </option>

      </select>

    </div>

    <div class="search-box">

      <input
        type="text"
        id="searchInput"
        placeholder="Search current page..."
      >

    </div>

    <button class="refresh-btn" onclick="loadStudents()">
      ↻ Refresh
    </button>

  </section>

  <!-- Table -->
  <section class="table-container">

    <div class="table-header">
```

Prionti_24BAI71005

```
        <div>
          <h2>Student Records</h2>

          <p id="showingText">
            Loading records...
          </p>
        </div>

      </div>

      <div class="table-wrapper">

        <table>

          <thead>

            <tr>
              <th>ID</th>
              <th>Name</th>
              <th>Email</th>
              <th>Course</th>
              <th>Age</th>
            </tr>

          </thead>

          <tbody id="studentTable">
          </tbody>

        </table>

      </div>

    </section>

    <!-- Pagination -->
    <section class="pagination-section">
```

Prionti_24BAI71005

```
        <button
          id="previousBtn"
          onclick="previousPage()"
        >
          ← Previous
        </button>

        <div id="pageNumbers" class="page-numbers">
        </div>

        <button
          id="nextBtn"
          onclick="nextPage()"
        >
          Next →
        </button>

      </section>

      <!-- Footer -->
      <footer>

        <p>
          Experiment 2.2.1 |
          Spring Boot Pagination & Sorting API
        </p>

      </footer>

    </div>

  <script src="script.js"></script>

</body>

</html>
```

Prionti_24BAI71005

7. OUTPUT:

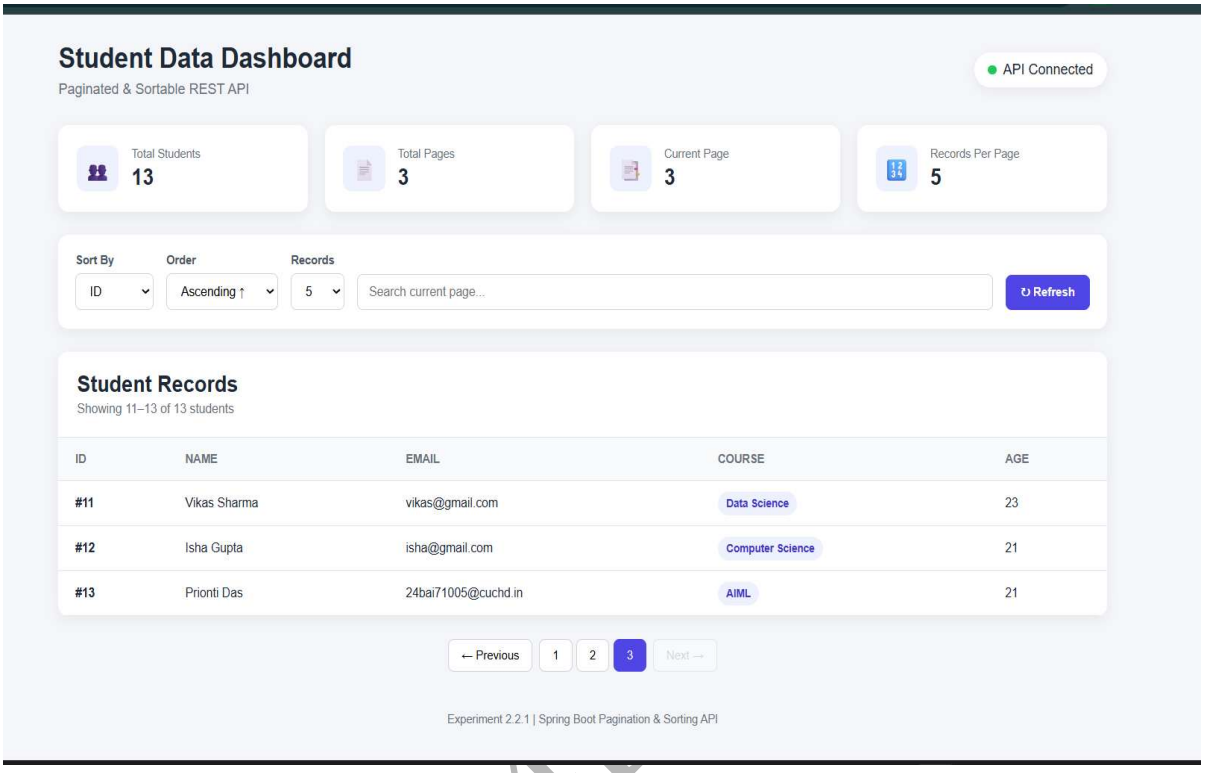


Fig 6.1: Student Data Dashboard – Paginated and Sortable REST API

The output shows a Student Data Dashboard that retrieves student records using a paginated and sortable REST API. It displays the total students, total pages, current page, and records per page. Users can select the sorting field, sorting order, and page size, search the current page, refresh the data, and navigate between pages using the pagination controls.

8. LEARNING OUTCOME:

1. Implement pagination in Spring Boot REST APIs using `Pageable` and `Page`.
2. Implement sorting based on different fields and sort directions.
3. Retrieve large datasets efficiently using page size and page number parameters.
4. Develop scalable and efficient REST APIs for data retrieval.
5. Test and verify pagination and sorting functionality through a web interface.

Prionti_24

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 6.2

1. AIM:

To optimize backend read operations using caching and efficient database query strategies.

2. OBJECTIVE:

1. To understand the concept and importance of caching in backend applications.
2. To implement caching mechanisms for frequently accessed data.
3. To reduce repeated database queries and improve application performance.
4. To implement efficient database query strategies for faster data retrieval.
5. To understand cache retrieval, storage, and eviction concepts.
6. To improve response time for backend read operations.
7. To reduce database load by serving frequently requested data from the cache.
8. To develop efficient and scalable backend applications using caching and optimized database queries.

3. SOFTWARE REQUIREMENTS:

1. Java JDK 17
2. Spring Boot
3. Visual Studio Code
4. H2 Database / MySQL

4. THEORY:

Caching is a technique used to temporarily store frequently accessed data so that future requests can be served faster without repeatedly accessing the database. This helps reduce database load and improves application response time.

Spring Boot provides caching support through annotations such as `@EnableCaching`, `@Cacheable`, `@CachePut`, and `@CacheEvict`. Frequently requested data can be stored in a cache and reused for subsequent requests.

Efficient database query strategies involve retrieving only the required data, avoiding unnecessary queries, and using optimized repository methods. Combining caching with efficient database queries improves backend performance, reduces database operations, and helps build scalable applications.

5. IMPLEMENTATION/PROCEDURE:

1. Create a Spring Boot backend application for optimizing database read operations and improving application performance.
2. Configure the project with required dependencies such as Spring Web, Spring Data JPA, H2/MySQL Database, and Spring Cache.
3. Create a Student entity containing fields such as ID, Name, Email, Course, Age, and Department.
4. Create a Student Repository using Spring Data JPA to perform database operations.
5. Create a Student Service to handle student retrieval and database-related business logic.
6. Implement a normal JPA query to retrieve student records and observe its execution performance.

Prionti_24BAI71005

7. Implement JOIN FETCH queries to retrieve related student data efficiently and reduce unnecessary database queries.
8. Implement native SQL queries for optimized database-level data retrieval.
9. Enable caching in the Spring Boot application using `@EnableCaching`.
10. Apply `@Cacheable` to frequently accessed student retrieval methods so that repeated requests can be served from the cache.
11. Implement a cache-clearing mechanism using `@CacheEvict` to remove stored data when required.
12. Develop a Backend Performance Optimization dashboard to display student count, cache status, and the optimization technique being used.
13. Add Query Performance controls for testing and comparing:
    - Normal Query
    - JOIN FETCH
    - Native SQL
14. Add a Cache Testing section with a student ID input field and Get Student and Clear Cache buttons.
15. Display the retrieved student's details such as Name, Email, Course, Department, and Response Time.
16. Display the complete Student Data in a table containing ID, Name, Email, Course, Age, and Department.
17. Measure and display the execution time of database queries to compare their performance.
18. Test the same student request multiple times to observe the effect of caching on response time.
19. Clear the cache and repeat the request to compare the cached and non-cached execution times.
20. Compare the performance of Normal Query, JOIN FETCH, and Native SQL operations and verify that optimized queries and caching reduce unnecessary database operations.
21. Run the application and verify that the dashboard displays the student data, cache status, query performance, and execution time correctly.
22. Capture the final dashboard output showing the Backend Performance Optimization, cache testing, query comparison, and student data.

6. CODE:

```
package com.prionti.performance_optimization.service;

import com.prionti.performance_optimization.dto.StudentRequest;
import com.prionti.performance_optimization.dto.StudentResponse;
import com.prionti.performance_optimization.entity.Department;
import com.prionti.performance_optimization.entity.Student;
import com.prionti.performance_optimization.repository.DepartmentRepository;
import com.prionti.performance_optimization.repository.StudentRepository;

import org.ehcache.Cache;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
```

Prionti_24BAI71005

```
import java.util.ArrayList;
import java.util.List;

@Service
public class StudentService {

    private final StudentRepository studentRepository;
    private final DepartmentRepository departmentRepository;
    private final Cache<Long, String> studentCache;

    public StudentService(
            StudentRepository studentRepository,
            DepartmentRepository departmentRepository,
            Cache<Long, String> studentCache) {

        this.studentRepository = studentRepository;
        this.departmentRepository = departmentRepository;
        this.studentCache = studentCache;
    }

    // ----------------------------------------------------
    // ADD STUDENT
    // ----------------------------------------------------

    @Transactional
    public StudentResponse addStudent(StudentRequest request) {

        Department department = departmentRepository
                .findById(request.getDepartmentId())
                .orElseThrow(() -> new RuntimeException(
                        "Department not found with ID: "
                                + request.getDepartmentId()
                ));

        Student student = new Student();

        student.setName(request.getName());
        student.setEmail(request.getEmail());
        student.setCourse(request.getCourse());
```

Prionti_24BAI71005

```
        student.setAge(request.getAge());
        student.setDepartment(department);

        Student savedStudent = studentRepository.save(student);

        return toResponse(savedStudent);
    }

    // ------------------------------------------------------------
    // NORMAL QUERY
    // ------------------------------------------------------------

    @Transactional(readOnly = true)
    public List<StudentResponse> getStudentsNormal() {

        List<Student> students =
                studentRepository.findStudentsNormal();

        List<StudentResponse> result = new ArrayList<>();

        for (Student student : students) {
            result.add(toResponse(student));
        }

        return result;
    }

    // ------------------------------------------------------------
    // OPTIMIZED QUERY USING JOIN FETCH
    // ------------------------------------------------------------

    @Transactional(readOnly = true)
    public List<StudentResponse> getStudentsOptimized() {

        List<Student> students =
                studentRepository.findStudentsWithDepartment();

        List<StudentResponse> result = new ArrayList<>();
```

Prionti_24BAI71005

```
        for (Student student : students) {
            result.add(toResponse(student));
        }

        return result;
    }

    // ------------------------------------------------------------
    // NATIVE SQL QUERY
    // ------------------------------------------------------------

    @Transactional(readOnly = true)
    public List<StudentResponse> getStudentsNative() {

        List<Object[]> rows =
                studentRepository.findStudentsUsingNativeQuery();

        List<StudentResponse> result = new ArrayList<>();

        for (Object[] row : rows) {

            Long id = ((Number) row[0]).longValue();

            String name = (String) row[1];

            String email = (String) row[2];

            String course = (String) row[3];

            int age = ((Number) row[4]).intValue();

            Long departmentId =
                    ((Number) row[5]).longValue();

            String departmentName =
                    (String) row[6];

            result.add(
                    new StudentResponse(
```

Prionti_24BAI71005

```
                    id,
                    name,
                    email,
                    course,
                    age,
                    departmentId,
                    departmentName
                )
            );
        }

        return result;
    }

    // ------------------------------------------------------
    // GET STUDENT BY ID WITH CACHE
    // ------------------------------------------------------

    @Transactional(readOnly = true)
    public StudentResponse getStudentById(Long id) {

        String cachedStudent =
                studentCache.get(id);

        // CACHE HIT
        if (cachedStudent != null) {

            System.out.println(
                    "CACHE HIT - Student ID: " + id
            );

            return parseCachedStudent(cachedStudent);
        }

        // CACHE MISS
        System.out.println(
                "CACHE MISS - Student ID: " + id
        );
```

Prionti_24BAI71005

```
        Student student =
                studentRepository.findById(id)
                        .orElseThrow(() ->
                                new RuntimeException(
                                        "Student not found with ID: "
                                                + id
                                )
                        );

        StudentResponse response =
                toResponse(student);

        String cacheValue =
                response.getId() + "|" +
                response.getName() + "|" +
                response.getEmail() + "|" +
                response.getCourse() + "|" +
                response.getAge() + "|" +
                response.getDepartmentId() + "|" +
                response.getDepartmentName();

        studentCache.put(
                id,
                cacheValue
        );

        return response;
    }

    // ------------------------------------------------------
    // CLEAR CACHE
    // ------------------------------------------------------

    public void clearCache() {

        studentCache.clear();

        System.out.println("CACHE CLEARED");
    }
```

Prionti_24BAI71005

```
// ---------------------------------------------------------
// CONVERT STUDENT TO RESPONSE DTO
// ---------------------------------------------------------

private StudentResponse toResponse(
        Student student) {

    return new StudentResponse(
            student.getId(),
            student.getName(),
            student.getEmail(),
            student.getCourse(),
            student.getAge(),
            student.getDepartment().getId(),
            student.getDepartment().getName()
    );
}

// ---------------------------------------------------------
// CONVERT CACHE STRING TO RESPONSE DTO
// ---------------------------------------------------------

private StudentResponse parseCachedStudent(
        String value) {

    String[] data =
            value.split("\\|", -1);

    return new StudentResponse(
            Long.parseLong(data[0]),
            data[1],
            data[2],
            data[3],
            Integer.parseInt(data[4]),
            Long.parseLong(data[5]),
            data[6]
    );
}
```

Prionti_24BAI71005

}

7. OUTPUT:

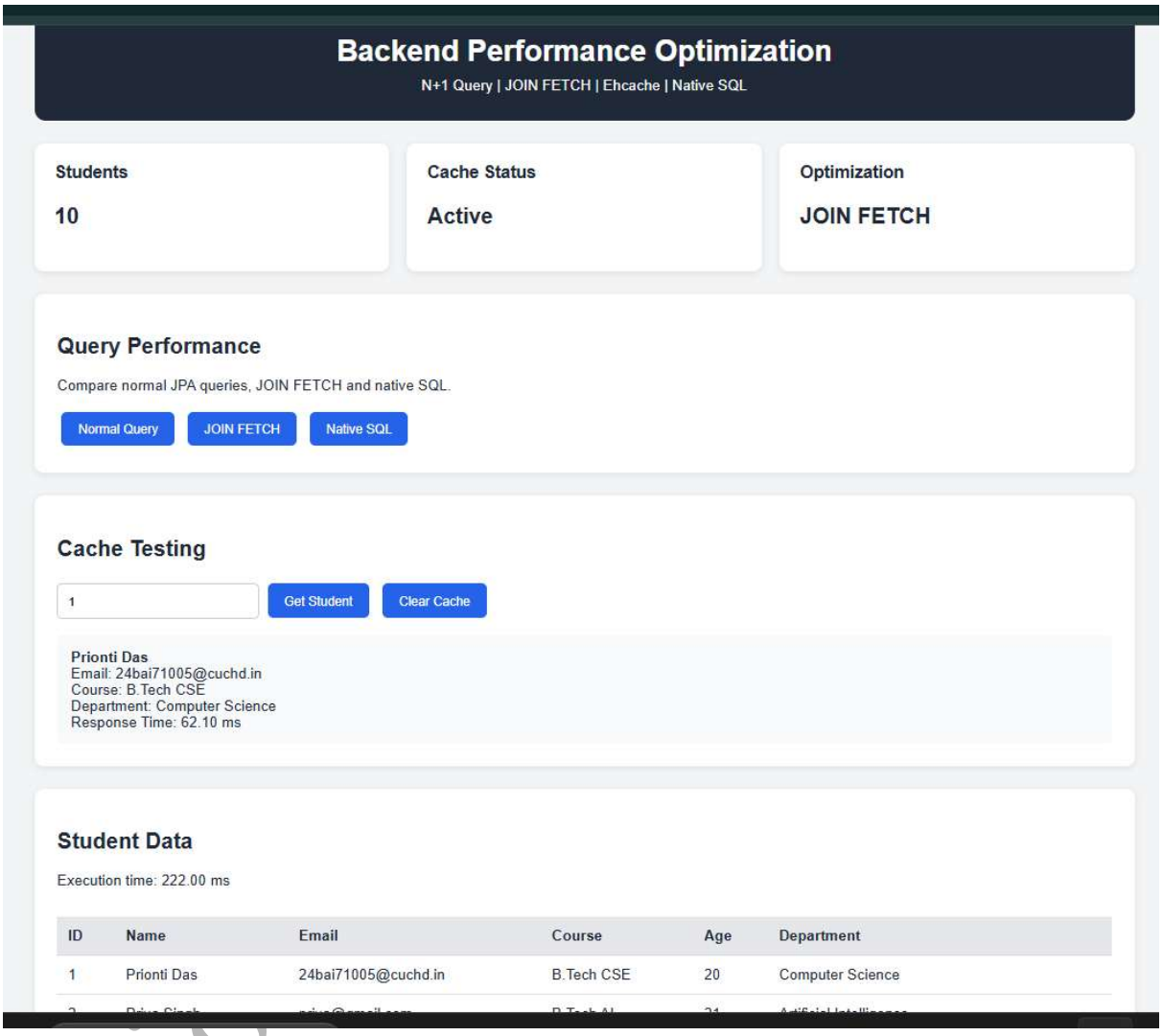


Fig 3.1: Student Performance Optimization Using Query Optimization and Caching

The experiment demonstrates **backend performance optimization** for student data retrieval. It uses different approaches such as a **normal database query, optimized `JOIN FETCH` query, native SQL query, and Ehcache-based caching**. The cache stores frequently requested student details so that repeated requests can be served faster without querying the database again. This improves **query efficiency, response time, and overall application performance**.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

8. LEARNING OUTCOME:

1. Understand different techniques for improving backend application performance.
2. Implement optimized database queries using `JOIN FETCH` and native SQL.
3. Use caching to reduce repeated database access and improve response time.
4. Compare normal and optimized data retrieval approaches.
5. Develop efficient and scalable backend services using Spring Boot

Prionti_24BAI71005